

1 soluție

1.1 Enunțul problemei

Se dă o matrice $n \times n$ cu valori întregi. Pornind din colțul stânga-sus $(1, 1)$ trebuie să ajungem în colțul dreapta-jos (n, n) , deplasându-ne doar la dreapta sau în jos. Pe lângă aceste mișcări standard, mai avem voie să facem o singură deplasare diagonală (fie $(i - 1, j + 1)$, fie $(i + 1, j - 1)$), care ne duce temporar în afara drumului principal. După diagonală trebuie să ne întoarcem pe drumul principal (prin simetrie) pentru a continua până la destinație. Scopul este să maximizăm suma valorilor vizitate.

1.2 Observații cheie

- Ne aflăm în fața unei probleme de tip **drum cu ocolire diagonală unică**. Sunt posibile două tipuri de ocoliri: una care urcă în diagonală (sus-dreapta) și alta care coboară în diagonală (jos-stânga).
- Din orice celulă (i, j) putem ajunge pe două căi:
 1. **Pe drumul principal** — din $(i - 1, j)$ sau $(i, j - 1)$, adică exact ca la problema clasică a drumului cu valori maxime.
 2. **Din cauza diagonalei** — am făcut deja ocolirea și acum ne întoarcem pe drumul principal. Convenim să notăm cu $dp1$ situația în care am venit prin diagonala sus-dreapta, iar cu $dp2$ situația în care am venit prin diagonala jos-stânga.
- Odată ce am părăsit drumul principal printr-o diagonală, **nu mai avem voie să-l părăsim a doua oară** — problema spune explicit că diagonală se face cel mult o dată. Astfel, $dp1$ și $dp2$ reprezintă stări „epuizate” în care opțiunea diagonalei a fost deja folosită.
- Procesăm celulele **în ordinea anti-diagonalelor** ($i + j$ constant). În acest fel, toate celulele necesare pentru calculul unei celule (i, j) au fost deja procesate, deoarece sursele (vecinii stânga, sus și cele două diagonale) au suma $i + j - 1$ sau $i + j - 2$, mai mică decât $i + j$.

1.3 Definirea starilor DP

Folosim trei tablouri DP de dimensiune $(n + 2) \times (n + 2)$:

- $dp[i][j]$ — valoarea maximă a drumului principal până în (i, j)
- $dp0[i][j]$ — $dp[i][j]$ pentru iterația curentă (actualizat temporar)
- $dp1[i][j]$ — drum care a folosit deja diagonala $(i - 1, j + 1)$
- $dp2[i][j]$ — drum care a folosit deja diagonala $(i + 1, j - 1)$

1.4 Relații de recurență

Pentru fiecare anti-diagonală $s = 3, 4, \dots, 2n$ procesăm toate celulele (i, j) cu $i + j = s$:

1. Pasul 1 — drum principal proaspăt:

$$dp0[i][j] = \max(dp[i-1][j], dp[i][j-1]) + m[i][j]$$

Se vine din $(i-1, j)$ sau $(i, j-1)$, fără să fi folosit diagonală.

2. Pasul 2 — drum cu ocolire sus-dreapta:

$$dp1[i][j] = \max(dp1[i-1][j+1], dp0[i-1][j+1]) + m[i][j]$$

Se vine din diagonală $(i-1, j+1)$. De acolo puteam avea fie un drum deja cu diagonală ($dp1$), fie un drum principal proaspăt ($dp0$) care tocmai a calculat valoarea pentru $(i-1, j+1)$.

3. Pasul 3 — drum cu ocolire jos-stânga:

$$dp2[i][j] = \max(dp2[i+1][j-1], dp0[i+1][j-1]) + m[i][j]$$

Analog, se vine din diagonală $(i+1, j-1)$. Surse posibile: stare „epuizată” ($dp2$) sau drum principal proaspăt ($dp0$).

4. Pasul 4 — agregare:

$$dp[i][j] = \max(dp0[i][j], dp1[i][j], dp2[i][j])$$

Inițializarea este $dp[1][1] = dp0[1][1] = dp1[1][1] = dp2[1][1] = m[1][1]$.

1.5 Corectitudinea soluției

- **Lema 1.** La pasul s al buclei, toate valorile dp , $dp0$, $dp1$, $dp2$ pentru celulele cu $i + j < s$ sunt deja finale.
- **Lema 2.** $dp[i][j]$ este valoarea maximă a unui drum valid de la $(1, 1)$ la (i, j) care respectă restricția „cel mult o diagonală”.
- **Lema 3.** $dp1[i][j]$ (respectiv $dp2[i][j]$) este valoarea maximă a unui drum care a trecut prin diagonală $(i-1, j+1)$ (respectiv $(i+1, j-1)$) și ajunge în (i, j) .
- **Dovada prin inducție pe anti-diagonale.** Baza este $(1, 1)$, evident corectă. Pasul inductiv folosește exact relațiile de recurență descrise, unde fiecare sursă provine dintr-o celulă cu sumă mai mică a indicilor (deci deja procesată). Prin maximizarea tuturor surselor legale obținem valoarea optimă pentru celula curentă. $\square$

1.6 Complexitate

- Bucla exterioară parcurge $2n - 1$ anti-diagonale.
- Pentru fiecare anti-diagonală iterăm $O(n)$ celule.
- **Timp:** $O(n^2)$
- **Memorie:** patru tablouri de dimensiune $(n + 2)^2$, adică $O(n^2)$. Cu $n \leq 1000$, memoria consumată este sub $4 \times 1005 \times 1005 \times 4$ octeți ≈ 16 MB.

1.7 Codul sursă

```
#include <bits/stdc++.h>
#define INF 1000000001
#define L 1005
using namespace std;
ifstream fin("birocratie.in");
ofstream fout("birocratie.out");

int n;
int m[L][L], dp[L][L], dp0[L][L], dp1[L][L], dp2[L][L];

void board() {
    for (int i = 0; i < n + 2; i++)
        for (int j = 0; j < n + 2; j++)
            m[i][j] = dp[i][j] = dp0[i][j] = dp1[i][j] = dp2[i][j]
                = -INF;
}

int main() {
    fin >> n;
    board();
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= n; j++)
            fin >> m[i][j];

    dp[1][1] = dp0[1][1] = dp1[1][1] = dp2[1][1] = m[1][1];

    for (int antidiag = 3; antidiag <= 2 * n; antidiag++) {
        // Pas 1: drum principal proaspat
        for (int i = 1; i <= n; i++) {
            int j = antidiag - i;
            if (j < 1 || j > n) continue;
            dp0[i][j] = max({dp[i - 1][j], dp[i][j - 1]}) + m[i][j];
        }
        // Pas 2: ocolire sus-dreapta
        for (int i = 1; i <= n; i++) {
            int j = antidiag - i;
```

```

        if (j < 1 || j > n) continue;
        dp1[i][j] = max({dp1[i - 1][j + 1], dp0[i - 1][j +
            1]}) + m[i][j];
    }
    // Pas 3: ocolire jos-stanga
    for (int i = n; i > 0; i--) {
        int j = antidiag - i;
        if (j < 1 || j > n) continue;
        dp2[i][j] = max({dp2[i + 1][j - 1], dp0[i + 1][j -
            1]}) + m[i][j];
    }
    // Pas 4: agregare in dp
    for (int i = 1; i <= n; i++) {
        int j = antidiag - i;
        if (j < 1 || j > n) continue;
        dp[i][j] = max({dp0[i][j], dp1[i][j], dp2[i][j]});
    }
}

fout << dp[n][n] << "\n";
return 0;
}

```

1.8 Explicație suplimentară despre cod

- Funcția `board()` inițializează toate celulele cu $-\infty$ pentru a evita cazuri neprevăzute la margini.
- Cele patru bucle pe anti-diagonală corespund exact celor patru pași din recurența descrisă anterior.
- Variabila temporară `dp0` este necesară deoarece $dp1[i][j]$ trebuie să citească valoarea **proaspăt calculată** a lui $dp0[i - 1][j + 1]$. Scriem întâi toate $dp0$, apoi toate $dp1$ și $dp2$, și abia la final copiem înapoi în dp .

1.9 Concluzie

Problema se rezolvă elegant cu programare dinamică pe grilă, adăugând stări suplimentare care urmăresc dacă diagonală a fost deja folosită. Parcurgerea pe anti-diagonale asigură ordinea corectă de calcul, iar complexitatea $O(n^2)$ este perfect adaptată pentru $n \leq 1000$.